# LAPORAN BACK-END DEVELOPMENT
# TUGAS AKHIR


**DISUSUN OLEH :**

**NOUVAL LUTVIL ALIFIAN (21114022610035)**

**PROGRAM STUDI S-1 INFORMATIKA PSDKU KEBUMEN**

**FAKULTAS TEKNOLOGI INFORMASI DAN SAINS DATA**

**UNIVERSITAS SEBELAS MARET**


**2026**

```
// ==========================================
// 1. DATA SIMULASI DATABASE 5 TABEL RELASIONAL
// ==========================================
// Sesuai dengan skema relasi di foto tugas Anda (users -> orders -> order_items -> books -> categories)

let categories = [
    { id: 1, name: "Pemrograman" },
    { id: 2, name: "Fiksi" }
];

let books = [
    { id: 1, title: "Belajar Python", price: 85000, category_id: 1 }, // category_id terhubung ke categories
    { id: 2, title: "Laskar Pelangi", price: 75000, category_id: 2 }
];

let users = [
    { id: 1, name: "Budi Santoso", email: "budi@mail.com" }
];

let orders = [
    { id: 1, user_id: 1, total_price: 85000 } // user_id terhubung ke users
];

let order_items = [
    { id: 1, order_id: 1, book_id: 1, quantity: 1 } // terhubung ke orders dan books
];

// ==========================================
// 2. FUNGSI SIMULASI LAYANAN REST API (CRUD)
// ==========================================

// --- [GET] Ambil Semua Buku ---
function getBooks() {
    return {
        status: 200,
        message: "Berhasil mengambil semua data buku",
        data: books
    };
}
```

```
// --- [GET] Ambil Satu Buku Berdasarkan ID ---
function getBookById(id) {
    const book = books.find(b => b.id === id);
    if (!book) {
        return { status: 404, error: "Buku tidak ditemukan" }; // Status 404 sesuai foto ke-3
    }
    return { status: 200, message: "Buku ditemukan", data: book };
}

// --- [POST] Tambah Buku Baru ---
function postBook(body) {
    const { title, price, category_id } = body;

    // Validasi error jika field kosong (Status 400 sesuai ketentuan foto ke-3 & 5)
    if (!title || !price || !category_id) {
        return { status: 400, error: "Field title wajib diisi" };
    }

    const newBook = {
        id: books.length > 0 ? books[books.length - 1].id + 1 : 1,
        title: title,
        price: Number(price),
        category_id: Number(category_id)
    };
    books.push(newBook);

    return { status: 201, message: "Buku berhasil ditambahkan", data: newBook }; // Status 201
Created
}

// --- [PUT] Mengubah Data Buku ---
function putBook(id, body) {
    const bookIndex = books.findIndex(b => b.id === id);
    if (bookIndex === -1) {
        return { status: 404, error: "Buku tidak ditemukan" };
    }

    const { title, price, category_id } = body;
    if (!title || !price || !category_id) {
        return { status: 400, error: "Data input salah atau tidak lengkap" };
    }
```

```
    books[bookIndex] = { id, title, price: Number(price), category_id: Number(category_id) };
    return { status: 200, message: "Buku berhasil diperbarui", data: books[bookIndex] };
}

// --- [DELETE] Menghapus Data Buku ---
function deleteBook(id) {
    const bookIndex = books.findIndex(b => b.id === id);
    if (bookIndex === -1) {
        return { status: 404, error: "Buku tidak ditemukan" };
    }
    books.splice(bookIndex, 1);
    return { status: 200, message: `Buku dengan ID ${id} berhasil dihapus` };
}


// ==========================================
// 3. PROSES SIMULASI PENGUJIAN OTOMATIS (OUTPUT CONSOLE)
// ==========================================
console.log("=== MEMULAI SIMULASI PENGUJIAN REST API TUGAS AKHIR ===\n");

// Test 1: Jalankan GET ALL BOOKS
console.log("[TEST METHOD: GET /books]");
console.log(JSON.stringify(getBooks(), null, 2));
console.log("\n--------------------------------------------------\n");

// Test 2: Jalankan POST BOOK (Menambah Buku Baru - Sesuai Contoh Request Body Foto 5)
console.log("[TEST METHOD: POST /books]");
const requestBodyPost = {
    title: "Belajar Python",
    price: 85000,
    category_id: 1
};
console.log("Kirim Request Body:", JSON.stringify(requestBodyPost));
console.log("Hasil Response (201):");
console.log(JSON.stringify(postBook(requestBodyPost), null, 2));
console.log("\n--------------------------------------------------\n");

// Test 3: Jalankan PUT BOOK (Mengubah data buku ID 1)
console.log("[TEST METHOD: PUT /books/1]");
const requestBodyPut = {
    title: "Belajar Python Edisi Premium",
    price: 99000,
```

```
    category_id: 1
};
console.log("Hasil Response (200):");
console.log(JSON.stringify(putBook(1, requestBodyPut), null, 2));
console.log("\n------------------------------------------------\n");

// Test 4: Jalankan GET BOOK BY ID (Mencari ID yang tidak ada untuk memicu error 404)
console.log("[TEST METHOD: GET /books/999 - Memicu Error 404]");
console.log(JSON.stringify(getBookById(999), null, 2));
console.log("\n------------------------------------------------\n");

// Test 5: Jalankan DELETE BOOK (Menghapus buku ID 2)
console.log("[TEST METHOD: DELETE /books/2]");
console.log(JSON.stringify(deleteBook(2), null, 2));
console.log("\n------------------------------------------------\n");

// Lihat hasil akhir database setelah operasi CRUD
console.log("=== ISI DATABASE BUKU TERBARU SELESAI CRUD ===");
console.log(JSON.stringify(books, null, 2));
```